

PACE: Deployment-Time Scheduling Policy for Deadline-Aware, Adaptive Intermittent Inference

Abstract—We study deadline-aware neural network inference on intermittently-powered, energy-harvesting embedded systems where unpredictable energy availability and storage-capacitor variation threaten timing reliability. Prior scheduling policies expose an accuracy–latency trade-off space within adaptive models but typically rely on fixed configurations derived from design-time profiles. These approaches are brittle because they cannot revise decisions when conditions shift mid-inference or when deployed hardware differs from design-time assumptions. Our key insight is that meeting deadlines under intermittence requires an online scheduling policy that (1) selects a configuration from the joint width–depth space at inference start and (2) re-plans depth at layer boundaries using only online recharge observations and (3) avoids design-time energy profiling. We implement this as PACE, a lightweight, best-effort MCU-resident runtime policy on a TI MSP430 that learns a deployment-time mapping from recharge duration to work-per-recharge. Across 20 measured and replayed energy traces and 4 capacitor configurations, PACE reduces deadline misses by up to 25× and improves total accuracy by up to 24 percentage points compared to design-time–profiled baselines.

I. INTRODUCTION

Intermittent, energy-harvesting (e.g., via solar or radio frequency harvesters) systems offer an appealing platform for long-term, maintenance-free sensing and neural network inference at the extreme low-power edge such as wildlife, structural, and health monitoring [1], [2], [3]. Local inference is a core requirement for such deployments: local computation consumes less energy than transmitting raw data [4], therefore extending operational lifetime, reducing communication cost, and enabling timely event detection under low-power regimes. However, these devices must operate under unpredictable environmental conditions [5], [6], most notably dynamic energy availability, making it difficult to execute long-running tasks (such as neural network inference) that require tight latency control. Applications such as object recognition, keyword detection, and event alerts often have fixed latency constraints [6], [7]; missing these deadlines degrades quality or causes mission failure [6], [8], [9], [10]. Ensuring accurate and deadline-aware inference under intermittent power and limited device memory remains a core challenge [10], [9], [11], [12], [13], [4], [14], [15], [16], [17]. In practice, harvested energy and storage capacitance, and inference time budget vary over time and across deployments, complicating deadline-aware execution.

Beyond energy volatility, deployment-time conditions can drift from design-time assumptions. For example, systems must tolerate storage-capacitor variation (tolerance/aging) [18], [19], [20], [21], causing further design- and deployment-time misalignment. As a result, scheduling policies derived from design-time profiles are brittle: when energy or capacitance differs at deployment, fixed choices can become late

(misses) or overly conservative (lost accuracy). For example, in our experiments a policy tuned on a 1.33 mF capacitor incurs a 0% miss rate at that capacitance but over 75% miss rate when the capacitor decays to 1.0 mF (§VII-B)—a variation well within normal manufacturing tolerance and aging [18].

To address the volatility of intermittent inference, recent neural models expose accuracy/latency *trade-off spaces* within a single network (preferred on Microcontroller Units (MCUs) due to limited memory). *Early exits* scale depth [22], [23], [24], [25] and *slimmable networks* scale width [26], [27]. SLEXNet combines both for even richer trade-offs [28]. We call the set of achievable accuracy–latency points the model’s *trade-off space*; each width–depth pair is a *configuration*.

Although these advances are an important step forward, they require a *scheduling policy* to select the most accurate, yet feasible configuration based on *unknown, time-varying deployment-time energy conditions*. Such a policy faces three unique challenges. First, it must **meet competing goals**, trading accuracy (deeper or wider configurations) against latency (shallower or slimmer configurations) [9], [10]. Second, it must **adapt to energy conditions that change mid-inference**, as intermittent inference often spans multiple recharge cycles [14], [4], [9], [10]. Finally, it must **adapt when deployment-time conditions do not match design-time conditions**. Most prior policies either (1) *greedily evaluate exits*, wasting compute on exits not taken [10], or (2) *pre-select a fixed configuration* at inference start using design-time profiles [28], [9], avoiding greedy waste but unable to adapt to mid-inference fluctuations or deployment/profile mismatches.

Therefore, intermittent inference under volatile energy and deployment-time capacitance variation requires a policy that (1) reasons over the configuration trade-off space, (2) dynamically re-plans inference when necessary, and (3) avoids design-time profiling. To this end, we design PACE (Progressive Adaptive Control Engine), which accomplishes our requirements with three core pieces: (1) an estimator that predicts end-to-end inference duration (compute plus recharge); (2) a recharge-aware configuration selection that searches over width–depth configurations at inference start; and (3) depth re-planning that switches to earlier exits only when necessary.

PACE’s **duration estimator** continuously analyzes task progression speed (i.e., average work completed per recharge) at deployment, associating distinct speeds with unique energy inflows (§IV-A). By relying only on online measurements, the duration estimation algorithm avoids design-time profiling at the cost of a brief, automated *deployment-time* warm-up (§IV-A). Continuous updates allow PACE to track changing energy inflows and capacitance variation without re-profiling. **Recharge-aware configuration selection determines current energy conditions based on recent recharge behavior**, esti-

mates duration using this recent behavior, and selects the most accurate configuration predicted to complete within the deadline (§IV-C). Finally, **dynamic depth re-planning** utilizes the duration estimation algorithm to *change model depth only when necessary* (as opposed to greedily), reducing deadline violations under scarce energy conditions and avoiding unnecessary computations under reliable energy conditions (§IV-D). While effective, these mechanisms are best-effort: unpredictable, stochastic energy conditions prevent accurate energy forecasting. Thus, PACE relies on continuous profiling and near-future energy proxies to reduce miss rates.

We implement and evaluate PACE using three MLPerf TinyML [29] applications under our intermittent safe, Just-in-Time (JIT) checkpointing runtime (§V-A). For each application, we deploy energy-efficient implementations of early exit and SLEXNet models (§V-B) across 20 measured and replayed volatile and non-volatile energy traces and 4 storage-capacitor configurations (§VI). PACE improves total accuracy by up to 24 percentage points and reduces deadline violations by up to 25 $\times$, with low memory overhead and no design-time profiling. Overall, we make the following contributions:

- **A best-effort deployment-adaptive scheduling policy for intermittent inference.** To our knowledge, PACE is the first intermittent inference scheduling policy that jointly reasons over a width–depth configuration space, re-plans depth at layer boundaries, and operates without design-time energy or hardware profiling, instead using a brief deployment-time warm-up to seed online estimators that are updated continuously thereafter.
- **Layer-boundary depth re-planning without greedy exit evaluation.** PACE re-plans depth only when online duration estimates indicate a deadline threat, rather than greedily evaluating every exit or committing to a fixed configuration. Ablation confirms that this mechanism and recharge-aware configuration selection each independently reduce deadline misses, and that their combination yields the lowest miss rate (§VII-D).
- **Adaptation to deployment-time capacitance variation without re-profiling.** PACE adapts to storage-capacitor changes—both decay and tolerance—within approximately 10 inferences, while design-time-profiled baselines incur up to 80% deadline miss rates under the same conditions (§VII-B).
- **Intermittent-safe MSP430 implementation.** We extend a JIT checkpointing runtime to support LEA-backed DNN execution and develop optimized SLEXNet kernels that reduce width-switching overhead by 2–10 $\times$.
- **Evaluation on real hardware across energy and capacitance conditions.** Across 80 configurations on a TI MSP430, PACE reduces deadline misses by up to 25 $\times$ and improves total accuracy by up to 24 percentage points, with less than 1% memory overhead.

PACE makes intermittent inference timeliness and accuracy less dependent on design-time assumptions about energy and hardware. After the anonymous review period we will release PACE and our evaluation artifacts as open source.

II. RELATED WORK AND SYSTEM MOTIVATION

Batteryless systems harvest ambient energy into storage-capacitors and execute in bursts between recharges, which complicates long-running tasks and deadline guarantees [8], [32], [33], [34], [35], [36], [37], [38], [39], [40], [30], [41], [42], [38], [43]. The core tension is that intermittence brings *unpredictable interruptions*, whereas deadline-aware execution assumes *predictable resources*. In practice: (1) harvested energy fluctuates with environment [2], [9], [10], [41], [5]; (2) useful tasks (e.g., DNN inference) span multiple power cycles and must meet deadlines that span several recharges [44], [14], [4], [45], [17]; and (3) storage-capacitors vary with tolerance [19], [20], [21] and aging [18]. These unpredictable deployment time changes create misalignment with design-time decisions. Scheduling policies that rely on design-time profiling are brittle to unexpected energy fluctuations and storage-capacitor variation: every change forces re-profiling and can lead to missed deadlines or wasted energy.

Given these challenges, a robust (i.e., deployment-agnostic) inference system on embedded devices must address, (a) how to construct neural network models that offer a rich range of accuracy/latency/energy tradeoffs, (b) how to schedule inferences to meet deadlines without relying on design-time profiling and assumptions (which may not remain valid in deployment), and (c) how to safely execute and degrade inference under frequent, intermittent power failures. This section surveys prior work in these areas, highlighting where their design-time assumptions can cause deployment-time problems. We then motivate the key principles behind PACE.

A. Embedded, Adaptive DNN Inference

To meet inference deadlines under unreliable energy, models must offer configurations that trade accuracy against latency and energy at varying granularity. Due to memory constraints on MCUs, we focus on three adaptive network families: **Early Exits** [22], [23], **Slimmables** [26], [27] and **SLEXNet** [28].

Early exits vary network *depth* by adding intermediate exits; later exits are more accurate but cost more time/energy [24], [46], [23]. Exits can be input-driven (conditional on input features) or resource-driven (determined by available time or energy) [10], [9]. Resource-driven execution can be *greedy* (evaluating each exit in succession) or *planned* (directly going to one exit determined at inference start). Greedy execution tolerates mid-inference fluctuations but wastes compute on exits that are not taken and adding exits increases per-exit overhead [24]. This paper focuses on resource-driven early exits: our policy re-plans at layer boundaries based on online recharge observations, rather than routing computation based on input features. Input-driven approaches (including mixtures-of-experts) cannot address mid-inference energy volatility, as their decisions are resource-independent and cannot react to subsequent power changes.

Slimmables vary *width* by enabling a subset of channels per layer [26], [27]. This yields finer trade-offs (Fig. 1), but the width is chosen at start and cannot be revised mid-run.

SLEXNet [28] varies both depth and width within a *single model*. On our TinyML workloads (§VI), SLEXNet

TABLE I: Comparison of Prior Work on Deadline-Aware Intermittent Inference.

	<i>System</i>	<i>Neural Network</i>		<i>Degradation Support</i>		<i>Capacitance Variation</i>
		<i>Architecture</i>	<i>Trade-offs</i>	<i>Before Inference</i>	<i>Mid-Inference</i>	
General	CatNap[6]/Culpeo[30]	N/A	N/A	No Support for Long Tasks	Not Supported	Continuous Profiling
	Celibi[7]	N/A	N/A	Design-Time Profiling	Not Supported	Not Supported
	REHASH[31]	N/A	N/A	Design-Time Profiling	Not Supported	Not Supported
	ePerceptive[9]	Early Exits	Coarse	Design-Time Profiling	Not Supported	Not Supported
	Zygarde[10]	Early Exits	Coarse	Greedy Execution	Greedy Execution	Greedy Execution
	SLEXNet[28]	Early Exit + Width	Fine	Design-Time Profiling	Not Supported	Requires Re-profiling
Inference	PACE (ours)	Early-Exit (optional) Width	Fine	Recharge-aware Configuration Selection	Depth Replanning	Online Updates

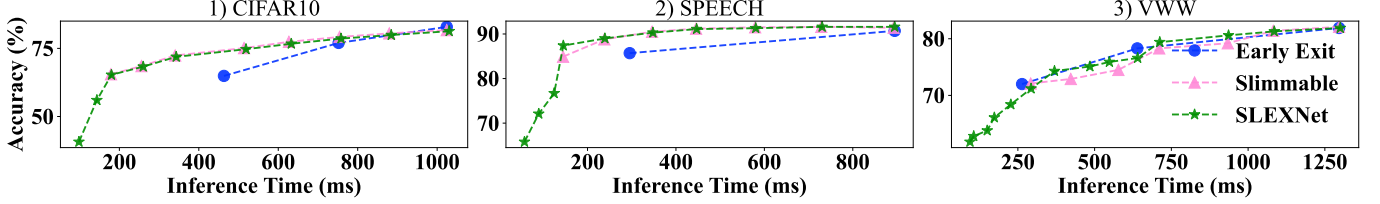


Fig. 1: Accuracy/Inference Time Pareto-frontier of Early Exit, Slimmable, and SLEXNet networks on constant power.

yields a denser Pareto frontier than Early-Exits and Slimmables (Fig. 1). These gains come with memory overheads: Early Exits add exit-specific weights, slimmables require per-width BatchNorm statistics [47], [27], and SLEXNet inherits the overheads of both (Table III).

B. Design-time, Profile-Guided Scheduling

To meet deadlines under unreliable power, prior work uses profile-guided scheduling and task degradation policies to pick a cheaper configuration. These policies work well *if* profiles match deployment, but most (1) select a fixed configuration at start and cannot adapt mid-run, and (2) require extensive *design-time* profiling that is brittle to deployment drift (e.g., capacitor variation, temperature, energy changes). We categorize prior work into general and inference-specific policies.

a) General Policies: Several general (i.e., not designed for DNNs) policies support deadline-aware execution. CatNap [6] and Culpeo [30] profile tasks online and degrade quality or frequency to maintain deadlines, and their continuous profiling adapts to gradual capacitor aging. However, they only support deadlines for short tasks that complete within a single discharge, degrade tasks only prior to their start, and cannot compensate for sudden capacitance changes that outpace the profiler. Celibi [7] and REHASH [31] adapt to energy variation but rely on offline profiling or design-time heuristics respectively, assume constant capacitance, and cannot degrade tasks mid-inference to meet deadlines.

b) Inference-Specific Policies: **ePerceptive** [9], **Zygarde** [10], and **SLEXNet** [28] manage the accuracy/energy tradeoffs by altering the DNN configuration at runtime.

ePerceptive's policy uses the most recent recharge time and a design-time derived lookup table to select a DNN configuration. Since the table is generated using a distribution of recharge times at design-time (taking up-to hours to profile in our experiments), it must be regenerated if deployment-time capacitance, energy, or inference budget is unexpected. ePerceptive is also not robust against energy fluctuations during inference as the policy does not track inference progress.

Zygarde partitions the DNN into mandatory and optional segments. Mandatory parts run greedily; optional parts run when a design-time learned factor predicts sufficient energy before the deadline [10]. Because mandatory segments always run, Zygarde does not require design-time tables at run time and is less sensitive to capacitor variation; however, greedily executing mandatory work wastes energy that could have powered more accurate later segments, and the learned factor must be recomputed for new deployments.

SLEXNet pre-selects a DNN configuration based on design-time data and power profiles. However, SLEXNet's policy tolerates unpredictable video frame rates under constant energy conditions, not unpredictable energy conditions in an intermittent setting. Thus, it cannot be directly compared to other intermittent scheduling policies. Moreover, SLEXNet's policy provides no mechanisms for mid-inference adjustments.

C. Motivation for PACE

The limitations of prior work reveal a clear gap: supporting deadline-aware, intermittent inference requires both adaptivity and robustness to deployment-time variability—in both hardware (storage-capacitor) and energy (distributions of energy availability). These limitations motivate a scheduling policy that (1) *reasons over the configuration trade-off space* at inference start, (2) *re-plans at layer boundaries* to handle mid-inference energy fluctuations, and (3) operates with *no design-time profiling* (just a brief deployment-time warm-up).

Table I summarizes the differences between PACE and prior work. Prior systems fall short along one or more of these axes. General scheduling policies either require offline, design-time profiling or do not support timeliness and degradation for long-running, multi-recharge tasks; they also do not support mid-task degradation. Inference-specific systems such as ePerceptive and SLEXNet rely on design-time profiles tied to a particular model and hardware configuration, so their decisions can become suboptimal when deployment conditions drift (e.g., energy dynamics or storage-capacitor variation), leading to deadline misses or lost accuracy under a deadline.

In contrast, PACE learns from deployment-time recharge observations and re-checks feasibility during execution: it selects a recharge-aware configuration at inference start and re-plans depth at layer boundaries when necessary. PACE applies to adaptive models that expose a configuration set and support resource-driven early termination (e.g., early exits), enabling mid-inference depth replanning; we evaluate PACE with Early-Exit and SLEXNet variants.

III. PACE: ASSUMPTIONS AND HIGH-LEVEL DESIGN

We introduce **PACE** (Fig. 2), an online, adaptive, deadline-aware policy for intermittent inference. PACE maintains high inference accuracy under both energy volatility and storage-capacitor variation by *avoiding design-time energy profiling*: it uses a brief *deployment-time warm-up* to seed online estimators and then adapts continuously from recharge observations.

PACE at a glance. At design time, developers provide (1) an adaptive neural network, and (2) a list of width-depth *configurations* sorted in order of decreasing accuracy (e.g., along the Pareto frontier). At deployment, PACE runs a short self-calibration phase (§IV-A) that measures the continuous-power inference duration for each configuration and seeds a lightweight, deployment-learned mapping used to predict inference duration under *unknown, and time-varying* deployment-time energy/recharge process $\mathcal{E}$. Empirically, warm-up converges within minutes in our RF-trace regimes; more generally, convergence depends on the number of recharge/update events observed during deployment and the number of configurations. Once warmed up, PACE operates under the following, best-effort policy:

- **Continuously during deployment:** PACE updates (1) an estimate of recharge duration and (2) a duration-estimation lookup table using *deployment-time measurements*. These updates track changes in energy conditions and storage-capacitor variation (tolerance/aging) without requiring any design-time energy or hardware profiles.
- **Before inference (recharge-aware configuration selection):** Using recent capacitor *recharge durations* as a proxy for incoming energy, PACE predicts inference duration for candidate *configurations* (width-depth pairs) and chooses the *most accurate* configuration expected to meet the given inference budget.
- **During inference (depth re-planning at layer boundaries):** After each layer, PACE re-estimates the predicted inference duration. If the prediction threatens to exceed the inference budget under updated energy observations, PACE *switches to an earlier exit* (shallower depth) to finish within the given budget.

Problem statement (best-effort under intermittence).

An inference job arrives at time t_0 with an *absolute* deadline $\mathcal{D}$ and an implied inference budget $\Delta = \mathcal{D} - t_0$. The model exposes a discrete set of configurations (width-depth pairs) ordered by accuracy. For any configuration, the inference duration under the continuous power is fixed regardless of scheduling policy (§IV), but the intermittent inference duration depends on an *unknown and time-varying* deployment-time energy/recharge process $\mathcal{E}$ (which determines recharge

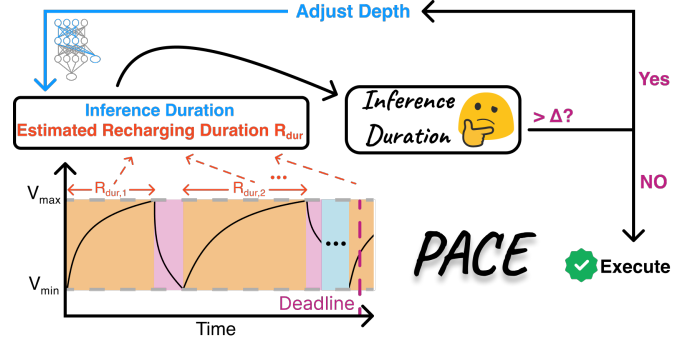


Fig. 2: Overview. PACE re-estimates duration mid-inference—degrading depth at layer boundaries to avoid deadline misses.

durations and how much work fits between recharges). We write the completion time as $T_{\text{dur}}(\pi; \mathcal{E})$, where π is an online policy that chooses an initial configuration and may revise it at layer boundaries based on observations. Ideally, one would seek to find a policy π that maximizes the inference accuracy (i.e., $A(\pi)$) while meeting the inference budget, i.e.,

$$\underset{\pi}{\operatorname{argmax}} A(\pi) \quad \text{s.t.} \quad T_{\text{dur}}(\pi; \mathcal{E}) \leq \Delta,$$

but $\mathcal{E}$ is not known a priori and can change mid-inference. PACE therefore implements a *best-effort* approximation: it uses recharge duration R_{dur} as an online proxy for $\mathcal{E}$, learns a deployment-time mapping $R_{\text{dur}} \mapsto R_{\text{W}}$ (work-per-recharge), and re-plans only when feasibility is threatened.

Core components. PACE comprises three components: (1) a **duration estimator** that predicts end-to-end inference duration (work + recharge), (2) **recharge-aware configuration selection** at inference start, and (3) **dynamic depth selection** at layer boundaries to prevent deadline misses.

Inference Duration estimation (§IV-B). Intermittent nodes harvest ambient, volatile energy into a storage capacitor and conduct work between recharges. We define work-per-recharge (R_{W}) as elapsed inference time under uninterrupted, continuous power occurring between recharges. PACE estimates the *duration of a single capacitor recharge* (R_{dur}) and uses it as a deployment-time signal of energy inflow. The estimator learns, *online*, the mapping between observed R_{dur} and R_{W} , and uses these to predict total inference duration T_{dur} , which includes all needed recharge and works-per-recharge ($T_{\text{dur}} = \sum R_{\text{dur}} + \sum R_{\text{W}}$). $\hat{R}_{\text{dur}}$, the estimation of R_{dur} , is updated after every recharge and the mapping from R_{dur} to R_{W} is updated whenever inference work occurs. These updates adapt the scheduling policy to *environmental shifts in energy* and *storage-capacitor variation* (tolerance/aging), improving accuracy and reducing misses.

Recharge-aware configuration selection (§IV-C). When an inference job arrives, PACE evaluates candidate configurations using the current energy estimate and selects the *most accurate configuration* expected to complete within the budget.

Depth re-planning (§IV-D). Not all fluctuations are predictable; if energy deteriorates mid-run, PACE re-plans at layer boundaries, compares the projected finish time to the remaining budget, and—only when necessary—*shortens depth*

TABLE II: Notation for PACE. $\dagger$: provided by developer; *: learned/measured at deployment; $\ddagger$: optional customization parameter. All other values are measured at runtime.

Notation	Definition
$P[]^\dagger$	Configurations (width–depth) sorted in decreasing accuracy
$\mathcal{D}^\dagger$	Absolute deadline timestamp (ms)
t_0	Task arrival time (ms)
Δ	$\mathcal{D} - t_0$, Inference budget (ms)
$T_{\text{ckpt}}^\ddagger$	Per-recharge checkpoint/recompute cost (ms)
$T_{\text{Bias}}^\ddagger$	Deadline margin for conservatism (ms)
$\mathcal{E}$	Unknown and variable deployment-time energy process
R_{dur}	Duration of a single recharge (ms)
R_W	Work-per-recharge: work done between recharges (ms)
T_W	$\sum R_W$, Work under continuous power (ms)
T_R	$\sum R_{\text{dur}}$, Total time spent recharging (ms)
$\#R$	Number of recharges needed to finish inference
$L[]^*$	Measured T_W for each configuration in $P[]$ (ms)
$\hat{R}_{\text{dur}}^*$	EWMA estimate of future recharge duration R_{dur} (ms)
$\hat{T}_{\text{dur}}$	$T_W + T_R$, Predicted inference duration (ms)
$\text{bin}(\cdot)$	Maps a recharge duration to one of 128 buckets
$M[]^*$	Deployment-learned mapping from recharge bucket to R_W : $M[\text{bin}(R_{\text{dur}})] \approx R_W$

by taking an earlier exit to avoid a miss. This policy avoids over-conservatism at start while preventing wasted work from greedily evaluating multiple exits.

Assumptions and Scope. PACE relies on: (1) Just-in-Time checkpointing so each layer has a stable work time under a fixed configuration (§V-A); (2) a timer that survives recharge (§V-A); (3) widths that are set at inference start (mid-run width changes are not supported in our setting; see §II-A); (4) multiple recharge events occur per inference (as seen in prior work [9], [6], [10], [14]) due to system capacitance limits;¹ and (5) the system is constantly charging irrespective of the workload or whether the system is on or recharging [5] (leading to varying rate of work completed per recharge under different energy conditions, §IV).

PACE targets *resource-driven* adaptation; input-dependent gating (conditional exits/MoE) is out of scope. PACE operates under unknown, stochastic energy resources and provides best-effort deadline miss reductions instead of probabilistic guarantees. Such guarantees require design-time modeling or profiling of energy conditions, which are time consuming [10] and contradict PACE’s no design-time energy profiling principles.

IV. PACE: DESIGN DETAILS

Utilizing our core insights, PACE forecasts near-future energy availability to enable best-effort intermittent inference before and during operation. Such forecasting relies on two exponentially weighted moving average (EWMA) algorithms to efficiently and accurately learn inference behavior online (§IV-E). Specifically, these estimators learn current energy conditions (i.e., recharge duration) and the progression speed

¹While larger capacitor sizes could result in one or no recharge events during inference, they would lead to increased recharge time and reduced reactivity to environmental events [41], [6].

Algorithm 1: The PACE Algorithm.

```

1 Function FirstBootWarmup():
    /* Measure  $T_W$ s at first boot. */
2   for Config in  $P[]$  do
3      $T_{W\text{-config}} \leftarrow \text{ProfileWork}(\text{Config})$ 
4     Append( $L[]$ ,  $T_{W\text{-Config}}$ )
5 Function RechargeUpdate( $R_{\text{dur}}$ ,  $R_W$ ):
    /* Apply updates after recharge. */
6    $\hat{R}_{\text{dur}} \leftarrow \alpha \cdot R_{\text{dur}} + (1 - \alpha) \cdot \hat{R}_{\text{dur}}$ 
7   if DuringInference() then
8      $M[\text{bin}(R_{\text{dur}})] \leftarrow \alpha \cdot R_W + (1 - \alpha) \cdot M[\text{bin}(R_{\text{dur}})]$ 
9 Function EstimateDuration( $T_W$ ):
10   $\#R \leftarrow \text{ceil}(T_W / M[\text{bin}(\hat{R}_{\text{dur}})])$ 
11  return  $\#R \cdot (\hat{R}_{\text{dur}} + T_{\text{ckpt}}) + T_W$ 
12 Function SelectConfiguration( $D$ ):
    /* Find Best Config within budget. */
13   $\Delta \leftarrow D - \text{CurrentTime}()$ 
14  for (Config,  $T_{W\text{-Config}}$ ) in Pair( $P[]$ ,  $L[]$ ) do
15    if EstimateDuration( $T_{W\text{-Config}}$ ) <  $\Delta - T_{\text{Bias}}$  then
16      return Config,  $T_{W\text{-Config}}$ 
17  return NoValidConfig, 0
18 Function RePlanDepth( $D$ , Config,  $T_{W\text{-Config}}$ ):
19  for Layer in Config do
20     $T_{W\text{-layer}} \leftarrow \text{ProfileWork}(\text{Layer})$ 
21     $T_{W\text{-Config}} \leftarrow T_{W\text{-Config}} - T_{W\text{-layer}}$ 
22     $\Delta \leftarrow D - \text{CurrentTime}()$ 
    /* Reconfigure Config if needed. */
23    if EstimateDuration( $T_{W\text{-Config}}$ ) >  $\Delta - T_{\text{Bias}}$  then
24      ReduceExit()

```

of inference under various energy conditions (i.e., work-per-recharge); these metrics are tracked continuously throughout deployment to adapt to changing circumstances (§IV-A). Combined (§IV-B), these two metrics can predict inference duration accurately before (§IV-C) and during inference (§IV-D) as shown in our evaluation (§VII).

The PACE Interface. Alg. 1 outlines PACE’s pseudocode; Table II lists notation. Developers provide an adaptive neural network and a list of configurations (width–depth pairs), sorted in order of decreasing accuracy along the Pareto frontier of the accuracy/duration trade-off space. Note that the Pareto frontier is a by-product of training such models; hence, there is no additional cost associated with this requirement [24]. PACE internally refers to this configuration list as $P[]$ (Table II). Developers must also invoke PACE with absolute inference deadline $\mathcal{D}$. At runtime, each task arriving at time t_0 is assigned an inference budget of $\Delta = \mathcal{D} - t_0$. T_{ckpt} and T_{Bias} are optional customization parameters; all other values in Table II are measured or learned.

Relative-time Semantics. PACE reasons in relative time. The estimator returns a predicted duration $\hat{T}_{\text{dur}}$ for a given work time T_W . PACE checks feasibility by comparing $\hat{T}_{\text{dur}}$

against the inference budget. The bias parameter T_{Bias} is applied as a *budget margin*: PACE treats a plan as feasible only if it is predicted to complete within $\Delta - T_{\text{Bias}}$.

Core Insights. PACE builds on three core insights. **First**, in a JIT checkpointing system such as Catnap [6], the work time spent on the same task (T_W) is constant regardless of energy inflow; only the time spent recharging (T_R) changes. As the number of recharges ($\#R$) increases, total task duration ($T_W + T_R$) increases due to the increase in T_R , not T_W .

Second, because energy also arrives during computation, the amount of time spent on work per recharge (R_W) is not constant: higher energy inflows yield larger R_W than lower energy ones. Given that T_W is constant and $T_W = \#R \times R_W$, lower energy inflows increase $\#R$ and thus, decrease R_W .

Third, the duration of a single recharge (R_{dur}) increases in low energy inflows and decreases in high energy ones. Hence, R_{dur} is a reasonable proxy to predict the deployment-time energy/recharge process $\mathcal{E}$ and has been used by Catnap, Culpeo, and REHASH [6], [30], [31]. ePerceptive [9] also uses the latest recharge duration to schedule inference tasks.

A. Learnable Components and Warm-up

Estimating Recharge Duration. PACE estimates $\hat{R}_{\text{dur}}$, a near-future estimate of recharge duration, based on past R_{dur} measurements, using an EWMA (Eq. 1). Subsection IV-E provides the rationale and parameter search for α .

$$\hat{R}_{\text{dur}} = \alpha \cdot R_{\text{dur}} + (1 - \alpha) \cdot \hat{R}_{\text{dur}} \quad (1)$$

Estimating Work-per-recharge. Given the insights, PACE predicts R_W from R_{dur} using a learned lookup table $M[\cdot]$ that maps recharge duration *buckets* to expected work-per-recharge. We discretize recharge durations into 128 buckets; let $\text{bin}(R_{\text{dur}})$ denote the bucket index for a measured or estimated R_{dur} . PACE updates the entry corresponding to the most recent observed recharge duration using an EWMA:

$$M[\text{bin}(R_{\text{dur}})] = \alpha \cdot R_W + (1 - \alpha) \cdot M[\text{bin}(R_{\text{dur}})] \quad (2)$$

Update Trigger. $\hat{R}_{\text{dur}}$ is updated after every recharge. In contrast, $M[\cdot]$ is updated when R_W is defined, i.e. only when inference has occurred between the prior two recharges. This design matches what the runtime can observe and avoids relying on stale R_W measurements when inference is infrequent.

Warm-up. $L[\cdot]$ (the T_W of each configuration in $P[\cdot]$),² $\hat{R}_W$, and R_{dur} are initially unknown. Given the first insight, $L[\cdot]$ requires measurement only once; in contrast, $\hat{R}_{\text{dur}}$ and R_W depend on $\mathcal{E}$ and hardware behavior (e.g., capacitance variation). PACE seeds unknown parameters at first boot. First, it executes all provided configurations ($P[\cdot]$), measures their T_W , and stores the results in $L[\cdot]$ (Alg. 1, lines 1–5). During these warm up runs, PACE also updates $\hat{R}_{\text{dur}}$ relevant $M[\cdot]$ entries (Alg. 1, lines 5–8). As described in update triggers, these estimates continue updating throughout deployment to track changing conditions.

²While T_W values are constant and can be measured at design-time, we measure them during the warm-up to avoid design-time calibration assumptions and to capture platform-specific costs.

B. Duration Estimation

PACE predicts $\hat{T}_{\text{dur}} = T_W + \#R \cdot (\hat{R}_{\text{dur}} + T_{\text{ckpt}})$ (Alg. 1 lines 9–11). To do so, it must estimate $\#R$ given a total work time T_W . Since $T_W = \#R \times R_W$ and $R_W \approx M[\text{bin}(\hat{R}_{\text{dur}})]$, PACE reasons that $\#R = \left\lceil \frac{T_W}{M[\text{bin}(\hat{R}_{\text{dur}})]} \right\rceil$. This routine is designed to provide *accuracy* and *efficiency* without any design-time profiling of energy inflows.

C. Recharge-Aware Configuration Selection

When an inference task arrives, PACE queries the duration estimator for all configurations in $P[\cdot]$ and chooses the most accurate configuration predicted to finish within the inference budget (Alg. 1, lines 12–17). PACE uses $\hat{R}_{\text{dur}}$ as a near-future estimate of recharge behavior. Under volatile energy inflows, this estimate provides a conservative initial setting that avoids assuming overly generous energy availability. If no feasible configurations exist, PACE reports a NoValidConfig status; applications need to determine the appropriate actions under such circumstances. We follow prior work and simply run the lowest-latency configuration.

D. Dynamic Depth Re-Planning

PACE avoids deadline violations by re-planning depth at each layer boundary (Alg. 1, lines 18–24). During inference, PACE tracks remaining work time by subtracting each layer’s work time from the configuration’s work time $T_{W-\text{Config}}$. After each layer, it re-queries the estimator using the updated $T_{W-\text{Config}}$ and current $\hat{R}_{\text{dur}}$. If the predicted completion time exceeds the budget, PACE degrades the model depth by switching to an earlier exit. This design avoids unnecessary execution of intermediate exits while producing schedules that are resilient to mid-inference energy fluctuations.

E. Rationale for EWMA and Parameter Search

PACE uses EWMA for two reasons. **First**, EWMA is a reasonable predictor of near-future (seconds-scale) recharge behavior because it is a low-pass IIR filter that down-weights older samples. This filters high-frequency noise while tracking stable trends. While longer-horizon predictors are possible, we find near-future prediction sufficient for accurate duration estimation (§VII-E). **Second**, Eqs. 1 and 2 can be implemented efficiently on the MSP430, which lacks floating point and fast division. With $\alpha = 2^{-N}$, EWMA updates can be performed using fixed-point arithmetic and shifts.

Parameter Search. The measured R_{dur} and R_W values used in Eqs. 1 and 2 are noisy, which impacts predicted duration. Accurate estimation depends on selecting α to filter noise while adapting to trends. We sweep five potential α values using CIFAR10 application under our experimental methodology (§VI). Each α test starts with an unpopulated lookup table and receives an identical warm-up budget (10 inference tasks per energy condition; §IV-A). After warm-up, we run an additional 10 inference tasks across 10 non-volatile energy conditions and record the latency, predicted latency, recharge count, and the predicted recharge count. Fig. 3 reports the percentage of over- and under-estimated recharge counts by PACE. We

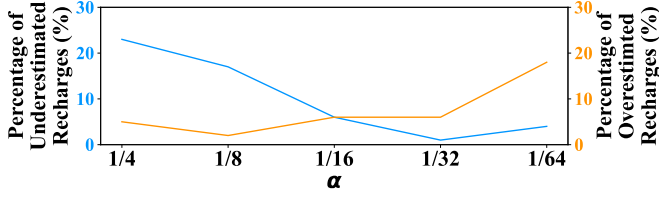


Fig. 3: Percentage of recharge under- and overestimation for a variety of potential α values.

find that larger α values underestimate more frequently while smaller α values overestimate more. Given the similarity of the neural network operation across underlying applications and since our measurements are averaged across a range of energy conditions, we select $\alpha = 2^{-5}$ as a practical trade-off for all experiments in this paper.

F. Customization Parameters

PACE can be customized using two values. **First**, the bias margin T_{Bias} controls how aggressively PACE degrades depth during inference (Alg. 1, lines 23–24). T_{Bias} guards against *underestimation* of inference duration, which arises from fixed-point arithmetic and systematic noise—properties of the platform and its implementation, not of the deployment environment. Under the same settings used to derive $\alpha = 2^{-5}$, 99% of inferences have a duration prediction bias above ≈ -20 ms. To avoid premature depth reduction, we set $T_{\text{Bias}} = 20$ ms as a fixed, platform-level default, analogous to α and the lookup-table size. Unlike the design-time energy profiles required by prior policies, this single setting suffices across all applications, traces, capacitances (§VII-B), and inference budgets (§VII-C) in our evaluation, and required no re-tuning as those conditions varied.

Second, the checkpointing computation cost T_{ckpt} extends PACE to non-JIT checkpointing systems. In our JIT runtime (§V-A), inference computation is not lost, and checkpointing overhead is incorporated into recharge measurements. In non-JIT systems, outages can cause variable recomputation [14], weakening the first insight. In such circumstances, T_{ckpt} can account for a conservative worst-case recomputation cost.

V. IMPLEMENTATION

A. Hardware and Runtime Support

PACE is implemented on a TI MSP430 Microcontroller Unit (MCU) [54], a popular platform in intermittent computing. The MSP430 provides 256 KB of FRAM, 8 KB of SRAM, up to 16 MHz CPU clock speed, and a Low Energy Accelerator (LEA) [55], but it provides no hardware support for floating-point or integer division. PACE exploits FRAM to preserve runtime state (e.g., estimator state and lookup tables) across recharge cycles without specialized remanence hardware.

From a software perspective, PACE integrates into the intermittent runtime through two lightweight hooks and one intermittent-safe execution mechanism: (i) a post-recharge handler that updates deployment-time estimators, (ii) an inter-layer callback that enables mid-inference depth re-planning, and (iii) JIT checkpointing support for CPU+LEA execution.

Post-Recharge Estimator Updates and Time-keeping.

PACE updates its recharge-duration estimate after each recharge event and updates the work-per-recharge mapping when inference work is observed between consecutive recharge events (§IV-A). We implement this logic in the runtime’s power-on initialization sequence, immediately after state restoration from a JIT checkpoint (see below).

For time-keeping, PACE uses the MCU’s internal clocks during both charging and discharging phases. As long as no true power failure occurs, R_{dur} and R_W can be tracked without a remanence time-keeper. In cases of true power failure, the current implementation conservatively assumes the inference deadline as missed. This did not occur in our experimental evaluation (over 10K individual inferences).

Inter-layer Callback for Mid-Inference Re-Planning.

To enable mid-inference adaptation, we instrument the DNN execution engine to invoke a callback at each neural-network layer boundary. This callback queries PACE’s duration estimator using the remaining work time of the current configuration and compares the projected duration to the remaining inference budget ($\Delta - T_{\text{Bias}}$). If the projection exceeds the margin, the callback triggers `ReduceExit()`, switching execution to a shallower exit and preserving timeliness. This callback-based integration keeps the scheduling logic separate from model kernels and adds only constant-time overhead work per layer.

Intermittent-safe Execution. Using insights from CatNap [6], PACE ensures safe execution via Just-in-Time (JIT) checkpoints. We extend CatNap’s JIT support by enabling checkpointing of the Low Energy Accelerator (LEA) during inference, requiring no explicit checkpointing inside DNN kernels. When the MCU’s internal comparator detects critically low voltage (V_{suspend}), the system checkpoints CPU and LEA state to FRAM and enters a low-power sleep mode until sufficient energy returns (V_{resume}), at which point it restores state and resumes inference. This avoids inference progress loss across outages and yields consistent per-recharge overhead. V_{suspend} and V_{resume} can be adjusted to control recharge granularity and the frequency of R_{dur} and R_W observations.

B. Accelerated Inference and Kernel Optimization

We extend Lupe’s DNN execution engine [14] to support the dynamic structural requirements of early-exit and SLEXNet models. A central challenge in MCU-class intermittent inference is the overhead of supporting width switching in SLEXNet variants, where different widths require different normalization parameters during execution.³

Width Support for SLEXNet Kernels. During inference, intermediate SLEXNet activations must be normalized using width-dependent BatchNorm statistics [27]. While these operations are inexpensive on GPUs [26], they incur substantial overhead on the MSP430 due to the lack of efficient hardware support for shifting/scaling and the relatively large activation feature maps. To reduce this overhead, we instrument Lupe’s

³While porting early-exit and SLEXNet to the LEA is not a main intellectual contribution of the paper, we view it as essential for fair evaluation. As we will release all the code open-source, we also view it as a community contribution, so others can build off this work.

TABLE III: Application details. Inference duration is the runtime of the most accurate tradeoff point. Memory required for inference is expressed as both raw value (kB) and percentage increase from the basic, non-adaptive model.

Inference Application	CNN Architecture	Width Mod. Ratio	Inference Duration (ms)	Exits	Widths	Inference Memory (kB) & Overhead (%)		
						Early Exit	Slimmable	SLEXNet
CIFAR10[48]	ResNet[49]	$\times 0.625$	1025	3	8	204 (48%)	142 (3%)	210 (53%)
SPEECH[50]	Depthwise Separable[51]	$\times 1.0$	900	2	7	125 (38%)	100 (10%)	137 (50%)
VWV[52]	MobileNetV1[53]	$\times 0.5$	1300	3	7	100 (31%)	89 (17%)	113 (49%)

TABLE IV: Points of Comparison.

System	Architecture (Tradeoffs)	Scheduling Policy	Degradation Support		Capacitance Variation	Budget (Δ) Variation	Used in Experiments
			Before Inference	Mid-Inference			
ePerceptive[9]	Early Exits (Coarse)	Lookup Table	Yes	No	No	No	QA,B,C
Zygarde[10]	Early Exits (Coarse)	Greedy	Yes	Yes	Yes	Yes	QA,B,C
SLEXNet[28]	SLEXNet (Rich)	Lookup Table	Yes	No	No	No	QA,B,C
PACE-E (ours)	Early Exit (Coarse)	PACE	Yes	Yes	Yes	Yes	QA,B,C
PACE-S (ours)	SLEXNet (Rich)	PACE	Yes	Yes	Yes	Yes	QA,B,C,D,E

kernels to apply scaling on kernel weights rather than activation feature maps. Depending on the model architecture, this optimization reduces the number of arithmetic operations required for width-specific normalization by $2\text{--}10\times$.

LEA-compatible scaling. We further normalize per-layer scaling factors so that weight adjustments can be accelerated by the MSP430’s LEA, avoiding CPU-intensive shift operations during width changes. These adjustments are performed in the inter-layer callback before LEA execution begins, which keeps the added runtime cost of adaptive-width support small relative to layer execution time on the target MCU (§VII-D).

C. Model Training and Pareto Configuration Generation

PACE assumes that the model provides a set of width-depth configurations with an accuracy ordering. We train SLEXNet variants using Universally Slimmable Networks [27] techniques including width-privatized BatchNorm [47], the sandwich rule, and in-place distillation. A weighted loss jointly optimizes all subnetworks across all exits in a single training process. We use neural architecture search to select early-exit locations and loss weights, optimizing both accuracy and trade-off space richness. The resulting configurations are sorted by decreasing accuracy and provided to PACE as $P[]$.

VI. EXPERIMENTAL METHODOLOGY

A. Intermittent Testing Setup

We use a 1mF capacitor for energy storage and similar to prior work [6], [11], [7], [9], utilize the P2110 Powerharvester Receiver [56] as our energy harvester. To test capacitor changes, we use parallel capacitors to achieve 1.17, 1.25, and 1.33 mF of capacitance (measured experimentally). Similar to prior work [30], [9], we create an experimental test-bed to accurately simulate incoming energy and consistently replay a variety of volatile and non-volatile energy conditions. The incoming energy is emulated using a 3.2 voltage generated by a Digilent Analog Discovery 3 [57] in series with a digital potentiometer. Using this setup, we can emulate over 100 different volatile and non-volatile power traces. Non-volatile traces exhibit small energy fluctuations (2-5%), imitating real world harvesters [6]. Volatile traces, on the other hand, randomly fluctuate in a wide range (20-80%) to imitate arbitrary

changes to energy conditions analogous to those we have seen in our lab (e.g., when a person walks by an RF emitter). We run each experiment on 10 randomly selected volatile and 10 randomly selected non-volatile traces (20 traces overall). The MSP430 clock is set to 16MHz.

Trace Justification and Volatility. We focus on RF-emulated traces because they exhibit high-frequency power fluctuations (20–80% variance) on the same timescale as a single DNN inference, directly stressing PACE’s mid-inference re-planning logic. Although these traces mimic an RF harvester in a dynamic office environment [5], PACE’s learned mapping ($M[]$) depends only on observed recharge durations and is agnostic to energy-source physics. Deployments with more stationary recharge behavior over an inference window should require fewer mid-inference revisions.

B. Inference Applications

We use the MLPerf tiny benchmark suite [29] for evaluation (Table III).⁴ Since two of the three original networks require more memory than the MSP430 can support, we modify only the width of these networks to fit the device constraints. Table III lists the width modification ratios and the required inference memory (activation + model weights) and percentage increase from the basic, non-adaptive variant of each application.

Each application is trained as an Early Exit and SLEXNet networks (§V-C). For proper comparison, early exits are located similarly in both architectures. Networks are optimized via Stochastic Gradient Descent (SGD) for 250epochs, with a learning rate of 0.1, momentum weight of 0.9, batch size of 256, and weight decay of 10^{-4} on all non-BatchNorm parameters. We use a linear learning rate scheduler. The average accuracy of the 5 trainings is utilized in our results.

Deadline Provenance. Following the conventions of soft real-time intermittent systems [9], [28], we define a consistent inference budget Δ as twice the continuous-power latency of the most accurate configuration ($P[0]$). This choice represents a deployment-time expectation that the time spent recharging is roughly proportional to the time spent computing [10].

⁴Prior work [9], [10] uses a subset of the applications or datasets from the MLPerf Tiny benchmark suite.

TABLE V: Accuracy and deadline misses across benchmark applications under non-volatile traces. Best values of accuracy and misses are highlighted in **blue** and **orange** respectively.

System	T. Acc (%) $\uparrow$ /			O. Acc (%) $\uparrow$ /			Misses (%) $\downarrow$		
	CIFAR10			VWW			SPEECH		
ePerceptive	54.4	73.5	/ 30	74.3	77.0	/ 10	71.4	87.0	/ 20
Zygarde	49.6	66.6	/ 30	73.9	76.6	/ 10	70.9	86.3	/ 20
SLEXNet	73.2	73.2	0	76.9	76.9	0	88.7	88.7	0
PACE-E	54.4	73.5	/ 30	74.3	77.0	/ 10	71.4	87.0	/ 20
PACE-S	73.4	73.4	0	76.9	76.9	0	89.1	89.1	0

This constraint serves two purposes: (1) it ensures that adaptation is strictly necessary under volatile energy conditions, as the system lacks the temporal margin to execute full-depth models if harvesting rates drop; and (2) it allows for a direct comparison against prior profile-based policies [9], [28] which were evaluated under similar moderate real-time pressure. While Δ is a fixed parameter in our primary experiments, we also experimented with varied and random Δ (§VII-C).

C. Points of Comparison

Table IV lists all points of comparison and their attributes. We compare PACE against prior scheduling policies on both Early-Exit and SLEXNet model families along the dimensions discussed in Section II. As original implementations are neither available nor directly applicable to our intermittent setting, we implement best-effort recreations from published descriptions and validate their operation experimentally. To isolate scheduling behavior from implementation effects, all approaches are evaluated using the same runtime optimizations and checkpointing systems outlined in Section V. All points of comparison run the smallest possible configuration under infeasible circumstances.

ePerceptive [9] uses an Early Exit architecture and schedules tasks based on a developer populated recharge time lookup table for a fixed inference budget (twice the inference latency noted in Table III). ePerceptive is unable to adjust to variation in mid-inference energy, capacitance, or Δ . Zygarde [10] greedily executes the exits in an Early Exit architecture. At the cost of potentially unused computation, Zygarde can degrade tasks during inference and handles capacitance variation. As SLEXNet [28] relies on continuous power, the work's original scheduler is not applicable to intermittent settings. In our comparisons, we instead utilize ePerceptive's scheduling strategy to schedule SLEXNet configurations. We compare these efforts with PACE under both Early Exit and SLEXNet (dubbed PACE-E and PACE-S respectively). Compared to other approaches, PACE requires no design-time profiling and instead adapts online to deployment-time variation.

Tuning and Warm-up. The tuning process for ePerceptive and SLEXNet consumed 2–4 hours per inference budget and capacitance and is outlined as follows. We run all model configurations 10 times — sweeping across all non-volatile traces discussed earlier — and record the duration of inference. Respective lookup tables are then populated to maximize accuracy while avoiding deadline misses given a design-time inference budget. Note that the capacitor's initial charge at

inference start impacts the inference duration, occasionally exceeding and occasionally meeting the inference budget for a given configuration. In these circumstances, lookup tables are populated conservatively to avoid the sporadic deadline misses. Zygarde requires no profiling. We warm up PACE (§IV-A) before experiments by exposing it to a variety of energy inflows until $L[]$, $M[]$, and $\hat{R}_{dur}$ are populated (under 5 minutes in our RF-style trace regimes) and EWMA updates continue whenever inference is running. We use a mapping lookup table $M[]$ of size 128 and set $T_{Bias} = 20$ ms, $T_{ckpt} = 0$, and $\alpha = 2^{-5}$ for the EWMA estimators. Unless otherwise specified, warm-up and baseline tuning are performed on a 1 mF capacitor.

D. Performance Metrics

We report three metrics, all in percentage: (1) *Misses*, the percentage of deadline violations; (2) *T. Acc.* (Total Accuracy), which scores each timely inference by its test accuracy and substitutes random-guess accuracy for missed deadlines; and (3) *O. Acc.* (On-time Accuracy), computed identically but excluding missed inferences. Thus, T. Acc. captures end-to-end utility under deadlines (penalizing misses), while O. Acc. isolates model fidelity on successful inferences. Note that identical accuracy values occur only when two different schedulers choose exactly the same configuration; even at similar miss rates, inference accuracy may differ based on exact configurations chosen by each policy.

VII. EXPERIMENTAL EVALUATION

Based on the experimental methodology discussed in Section VI, we strive to answer the following questions (Qs),

- **QA – Overall Performance:** How does PACE's policy impact accuracy and deadlines misses in (**QA.1**) non-volatile and (**QA.2**) volatile energy conditions?
- **QB – Capacitance Variation:** How does PACE compare against prior policies under capacitance variation?
- **QC – Inference Budget Variation:** Can PACE and prior policies adapt to changes in inference budget?
- **QD – Degradation Ablation:** How do recharge-aware configuration selection and per layer depth re-planning impact accuracy and deadline misses of the PACE policy?
- **QE – Overhead, Warm-Up, and Accurate Estimates:**
 - **QE.1** What are the overheads of PACE ?
 - **QE.2** How much warm-up does PACE require? How quickly do learned values converge?
 - **QE.3** Can PACE accurately estimate $\hat{T}_{dur}$ and $\#R$?

A. Overall Performance

For each trace (10 non-volatile and 10 volatile), we conduct 100 inference tasks per application and point of comparison. We first compare policies under non-volatile traces (Table V) and then under volatile traces (Table VI); the volatile table reports both absolute values and changes relative to the non-volatile baseline. Note that since SLEXNet equipped models can operate in lower energy inflows than Early-Exit ones,

TABLE VI: Accuracy, deadline misses, and comparison to non-volatile results across benchmark applications under volatile traces. Best values of accuracy and misses are highlighted in **blue** and **orange** respectively.

System	CIFAR10			VWW			SPEECH		
	Total Acc (%) ↑	On-time Acc (%) ↑	Misses (%) ↓	Total Acc (%) ↑	On-time Acc (%) ↑	Misses (%) ↓	Total Acc (%) ↑	On-time Acc (%) ↑	Misses (%) ↓
ePerceptive	40.2(-14.2) / 72.2(-1.3) / 51.4(+21.4)	70.8(-3.5) / 76.3(-0.7) / 20.7(+10.7)	59.1(-12.3) / 86.5(-0.5) / 35.4(+15.4)						
Zygarde	44.6(-5.0) / 67.1(+0.5) / 39.4(+9.4)	72.3(-1.6) / 75.9(-0.7) / 13.9(+3.9)	61.4(-9.5) / 86.0(-0.3) / 32.0(+12.0)						
SLEXNet	41.8(-31.4) / 68.6(-4.9) / 45.7(+45.7)	61.8(-15.1) / 76.5(-0.4) / 55.7(+55.7)	64.3(-24.4) / 87.4(-1.3) / 29.5(+29.5)						
PACE-E	45.8(-8.6) / 70.9(-2.6) / 41.3(+11.3)	72.3(-2.0) / 76.1(-0.9) / 14.6(+4.6)	61.9(-9.5) / 86.6(-0.4) / 31.9(+11.9)						
PACE-S	64.4(-9.0) / 69.4(-4.0) / 8.4(+8.4)	73.7(-3.2) / 74.2(-2.7) / 2.2(+2.2)	80.8(-8.3) / 86.0(-3.1) / 6.7(+6.7)						

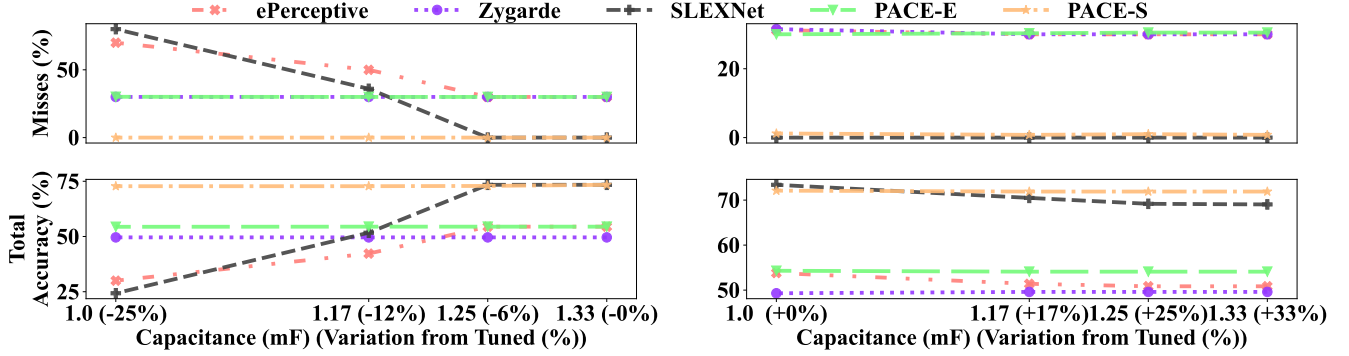


Fig. 4: Misses and total accuracy across capacitor values; all policies are tuned/warmed on 1.33 mF (left) and 1.0 mF (right). PACE adapts to decreasing (left) and increasing (right) capacitance while ePerceptive and SLEXNet operate sub-optimally under divergent capacitance values. Zygarde tolerates capacitance changes, but with lower total accuracy due to greedy execution.

Early-Exit equipped systems are likely to miss deadlines in low energy conditions, regardless of fluctuation volatility.

QA.1 : How does PACE’s policy impact accuracy and deadlines misses in non-volatile energy conditions? Under stable energy and when design-time and deployment-time conditions match, fixed-profile policies remain valid; PACE’s main benefit in this regime is matching their performance without requiring design-time profiling. PACE-E and PACE-S perform comparably to ePerceptive and SLEXNet, respectively, while avoiding design-time profiling and greedy execution (Table V). PACE-S shows a small advantage over the SLEXNet baseline even at 0% misses because the fixed lookup table must choose configurations safely across expected recharge behavior and possible capacitor charge states at inference arrival. Tuning more aggressively could improve accuracy but would risk misses under unfavorable starting capacitor conditions. PACE instead observes current recharge behavior and capacitor charge online and occasionally selects a more accurate configuration while still avoiding misses. Zygarde has worse total and on-time accuracy than either PACE-E or ePerceptive due to wasted work from greedily evaluating exits that are not ultimately taken.

QA.2 : How does PACE’s policy impact accuracy and deadlines misses in volatile energy conditions? Under volatile conditions—where energy fluctuates and deployment-time behavior diverges from design-time assumptions—PACE-S and PACE-E are both effective, adapting to volatile conditions without design-time profiling, reducing misses by up to 25×, and improving total accuracy by up to 24 percentage points (Table VI). PACE-S incurs the fewest deadline misses and has the highest total accuracy. This is due to a combination

of SLEXNet’s fine-grained tradeoff space and PACE’s ability to exploit this space under volatile conditions (e.g., PACE-S performs better than PACE-E despite using the same underlying policy). PACE-S does not always have the highest on-time accuracy because it intentionally selects earlier exits to avoid deadline misses, trading some on-time accuracy for higher total accuracy under volatile conditions. Finally, PACE-E still outperforms ePerceptive in both misses and total accuracy, while achieving comparable results to Zygarde.

B. Capacitance Variation

We conduct two unique experiments on the CIFAR10 application to evaluate the performance of scheduling policies under capacitance variation, separately testing decreased capacitance (decay) and increased capacitance (tolerance). In each experiment, we run 100 CIFAR10 tasks for each trace (10 non-volatile ones), capacitor option, and point of comparison.

Capacitance Decay We emulate the impact of decreased capacitance due to decay, which occurs throughout a capacitor’s lifetime. We first fine-tune ePerceptive and SLEXNet and warm-up PACE variants on a 1.33 mF capacitor. We then change the capacitor to 1.25, 1.17, and 1.0 mF consecutively. Misses and total accuracy are reported in Fig. 4 (left).

Capacitance Tolerance We emulate the impact of increased capacitance due to tolerance (reduced capacitance due to tolerance is addressed similarly as above). We fine-tune ePerceptive and SLEXNet on a 1 mF capacitor. PACE variants begin with a populated $L[]$ but an unpopulated $M[]$. We then change the capacitor to 1.17, 1.25, and 1.33 mF. Misses and total accuracy are reported in Fig. 4 (right).

TABLE VII: Accuracy and deadline misses of CIFAR10 application across a range of inference budgets (relative to duration of the most accurate configuration). Best values of accuracy and misses are highlighted in **blue** and **orange** respectively.

System	1.0×			1.5×			2.0×			3.0×			4.0×			6.0×			RNG [1.0-6.0×		
	Total Acc (%) ↑ / On-time Acc (%) ↑ / Misses (%) ↓																				
ePerceptive	10.0 / 0.0 / 100			10.0 / 0.0 / 100			54.4 / 73.5 / 30			59.9 / 72.4 / 20			62.7 / 72.0 / 15			70.9 / 70.9 / 0			52.0 / 71.7 / 32		
Zygarde	26.5 / 64.9 / 70			37.5 / 64.9 / 50			49.6 / 66.6 / 30			59.9 / 72.4 / 20			66.8 / 75.3 / 13			76.9 / 76.9 / 0			58.5 / 72.9 / 23		
SLEXNet	10.0 / 0.0 / 100			10.0 / 0.0 / 100			73.2 / 73.2 / 0			73.2 / 73.2 / 0			73.2 / 73.2 / 0			73.2 / 73.2 / 0			60.1 / 73.4 / 21		
PACE-E	26.5 / 64.9 / 70			40.5 / 70.9 / 50			54.4 / 73.5 / 30			63.9 / 77.4 / 20			69.2 / 78.9 / 14			78.7 / 78.7 / 0			61.9 / 77.5 / 23		
PACE-S	58.1 / 63.4 / 10			68.2 / 68.2 / 0			73.4 / 73.4 / 0			76.7 / 76.7 / 0			78.3 / 78.3 / 0			79.8 / 79.8 / 0			75.4 / 75.4 / 0		

QB : How does PACE compare against prior policies under capacitance variation? PACE maintains low miss rates and high total accuracy across capacitance changes without offline re-tuning. In both experiments, PACE’s policy converges within 10 inferences after a change in capacitance, including the partially warmed setting in Fig. 4 (right). In contrast, ePerceptive and SLEXNet degrade as deployment-time capacitance diverges from design-time profiles, incurring higher misses and/or lower total accuracy. Zygarde tolerates capacitance variation without re-profiling, but its greedy execution leads to lower total accuracy than PACE.

C. Inference Budget Variation

While our primary experiments use a consistent inference budget (Δ) for clear comparison with prior work, real deployments may face varying deadline pressure. To evaluate robustness across budgets, we run 100 CIFAR10 tasks for each trace (10 non-volatile ones), budget option, and point of comparison. Total accuracy is reported in Tab. VII.

QC : Can PACE and prior policies adapt to changes in inference budget? Yes, PACE-S achieves the highest total accuracy at every budget. Its advantage over baselines is most pronounced under tight deadlines—precisely the domain where scheduling policy matters most. At 1.0–1.5 $\times$, where the budget has little room for error, design-time–profiled policies (ePerceptive, SLEXNet) overestimate the feasible configuration and always miss, while PACE-S eliminates misses entirely by 1.5 $\times$ and maintains high on-time accuracy throughout. As budgets relax, all policies miss less. This behavior is expected—generous budgets reduce the value of any scheduling policy. Importantly, the tight-deadline regime is the most relevant to intermittent deployments: a system with 6 $\times$ its compute time is unlikely to be energy-constrained in practice. However, even in this relaxed operating regime, PACE demonstrates the highest accuracies as it correctly identifies the most accurate, yet feasible configurations. Other policies perform sub-optimally due to either wasted work (Zygarde) or sub-optimal configuration selection (ePerceptive and SLEXNet). Specifically, ePerceptive and SLEXNet were tuned for the 2 $\times$ budget and cannot exploit the extra slack, so they plateau. PACE recomputes feasibility against the actual current budget and therefore selects more accurate configurations as slack increases. Overall, these results validate that online, recharge-aware scheduling yields its greatest benefit when deadlines are binding and adaptation is essential.

D. Degradation Ablation

We analyze the impact of PACE’s recharge-aware configuration selection and per layer depth re-planning by running 100 CIFAR10 tasks for each trace (10 volatile ones) with 4 different version of PACE-S, described below. As a reference, under non-volatile traces PACE’s degradation strategies do not introduce measurable downsides (Table V). Under volatile traces, however, the same strategies substantially reduce deadline misses (Fig. 5 and Table VIII).

- **No Adjust** neither configuration selection nor depth re-planning are used and observed R_{dur} are utilized (instead of the EWMA estimate $\hat{R}_{\text{dur}}$).
- **Only Mid** only depth re-planning is used and observed R_{dur} are utilized (instead of the EWMA estimate $\hat{R}_{\text{dur}}$).
- **Only Before** only configuration selection is used.
- **PACE (Both)** the default PACE configuration, which enables both configuration selection and depth re-planning.

QD : How do recharge-aware configuration selection and per layer depth re-planning impact accuracy and deadline misses of the PACE policy? The default PACE configuration (PACE (Both)), which combines both mechanisms, achieves fewer deadline violations than either mechanism alone and the highest total accuracy (Table VIII). Recharge-aware configuration selection reduces deadline misses by preventing overly optimistic start-of-inference choices under volatile traces (compare **No Adjust** to **Only Before**). Per-layer depth re-planning further reduces misses by shortening depth when conditions deteriorate mid-inference (compare **No Adjust** to **Only Mid**). These results validate PACE’s core scheduling mechanisms: recharge-aware configuration selection at inference start and per-layer depth re-planning during execution.

E. Overhead, Warm-Up, and Accurate Estimates

QE.1 : What are the overheads of PACE? We profile 100 PACE-S inferences across a range of non-volatile energy conditions and applications to characterize PACE’s per-recharge and per-inference runtime overheads (Table IX). At each recharge, PACE updates $M[]$ and $\hat{R}_{\text{dur}}$ (§IV-A), incurring only 0.01 ms overhead per recharge due to efficient fixed-point EWMA updates (§IV-E). PACE queries the latency estimator at each layer boundary, which contributes a per-inference runtime overhead of less than 7.5 ms (including initial configuration selection and layer-boundary estimator queries). The majority of this overhead stems from the lack of a hardware divider on the MSP430. PACE incurs approximately 2.6 kB of total memory overhead. Of this, roughly 2.0 kB corresponds to

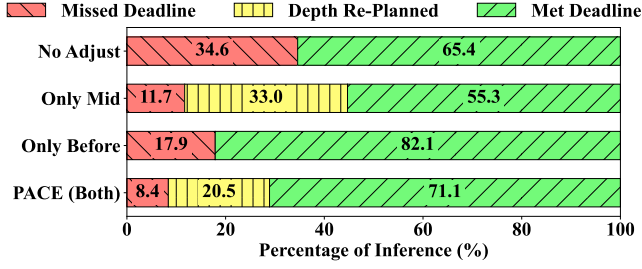


Fig. 5: Missed and successful inferences for four PACE variants (described in §VII-D) under volatile traces. Successful inferences are subdivided into those that meet deadlines using the initial configuration (green) and those that meet deadlines after depth re-planning (yellow). **PACE (Both)**—which combines recharge-aware configuration selection and per-layer depth re-planning—incur the fewest overall misses.

TABLE VIII: Accuracy and deadline misses of ablation study on the CIFAR10 application. Best values (lowest for misses, highest for accuracy) in each column are highlighted in blue.

System	CIFAR10		
	T. Acc (%) ↑	O. Acc (%) ↑	Misses (%) ↓
No Adjust	49.3	70.1	34.6
Only Mid	59.0	65.6	11.7
Only Before	60.5	71.5	17.9
PACE (Both)	64.4	69.4	8.4

the lookup table, estimator state, and related runtime values, while the remainder is text/code overhead.

The JIT checkpointing incurs overheads as well (Table IX). We find that JIT checkpointing at critical power levels incurs approximately 0.89 ms overhead per recharge, mostly due to copying LEA SRAM, the CPU stack, and CPU registers into non-volatile FRAM. These copies also incur a memory overhead of 5.3 kB (comparable to Lupe’s 4 kB of LEA SRAM). We note that all approaches incur this JIT overhead—it is not specific to PACE, but we include it for completeness. Overall, these overheads are small relative to inference latency and recharge durations in our workloads, supporting PACE’s utility as an online policy on MSP430-class devices.

QE.2 : How much warm-up does PACE require? How quickly do learned values converge? PACE requires a first-boot warm-up phase to populate L , M , and $\hat{R}_{dur}$ (§IV-A). In our RF-style trace regimes, this initialization completes in under 5 minutes, while M and $\hat{R}_{dur}$ continue to refine online during subsequent inference. Based on our experiments, M values converge within 5–10 inference tasks and $\hat{R}_{dur}$ converges within 5–10 recharges. Before convergence, the estimator behaves conservatively, preferring to avoid deadline misses (except in cases of insufficient power) at the cost of temporarily suboptimal accuracy.

QE.3 : Can PACE accurately estimate $\hat{T}_{dur}$ and $\#R$? We profile 100 PACE-S inferences across a range of non-volatile energy conditions and record the estimated $\hat{T}_{dur}$, predicted $\#R$, observed T_{dur} , and observed $\#R$. Overall, PACE’s estimates are accurate enough for scheduling: it predicts the number of required recharges correctly in 93% of inferences and exhibits

TABLE IX: Runtime and memory overhead of PACE and the JIT checkpointing. Memory overhead is reported in kB and as a percentage of on-device memory (256 kB).

Source	Per Recharge Overhead (ms)	Per Inference Overhead (ms)	Memory Overhead (kB)
PACE	0.01	< 7.5	2.6 (1%)
JIT	0.89	0	5.3 (2.1%)

a small positive duration bias of +21 ms.

VIII. CONCLUSION

PACE is a deployment-adaptive policy for best-effort, deadline-aware inference on intermittent MCU-class sensor nodes. PACE reasons over the configuration space, selects an initial configuration from recent recharge behavior, and re-plans depth at layer boundaries using online recharge observations, allowing the system to adapt to energy volatility and storage-capacitor variation without design-time energy profiling. Empirically, PACE reduces deadline misses by up to 25× and improves total accuracy by up to 24 percentage points across multiple adaptive model families, including Early-Exit and SLEXNet. Rather than relying on expensive and brittle design-time profiles, PACE uses a brief deployment-time warm-up to seed its estimators and continues learning online. By correcting course mid-inference rather than greedily evaluating exits, PACE avoids wasted computation and improves useful accuracy under volatile energy conditions. Broadly adopted, PACE could help intermittent sensors meet soft real-time requirements in batteryless deployments for tasks such as on-node detection and closed-loop control without site-specific tuning.

REFERENCES

- [1] M. Afanasov *et al.*, “Battery-less zero-maintenance embedded sensing at the mithræum of circus maximus,” in *SenSys ’20*, Nov. 2020.
- [2] H. Desai, M. Nardello, D. Brunelli, and B. Lucia, “Camaroptera: A Long-range Image Sensor with Local Inference for Remote Sensing Applications,” *ACM Trans. Embed. Comput. Syst.*, vol. 21, pp. 32:1–32:25, May 2022.
- [3] G. Chen *et al.*, “A cubic-millimeter energy-autonomous wireless intraocular pressure monitor,” in *2011 IEEE International Solid-State Circuits Conference*, Feb. 2011, pp. 310–312.
- [4] G. Gobieski, B. Lucia, and N. Beckmann, “Intelligence Beyond the Edge: Inference on Intermittent Embedded Systems,” in *ASPLOS ’19*, Apr. 2019, pp. 199–213.
- [5] P. M. Gholami, M. Xiang, and H. Hoffmann, “Position paper: Voltage change is not energy consumption,” in *ENSys ’25*, 2025, p. 33–35.
- [6] K. Maeng and B. Lucia, “Adaptive low-overhead scheduling for periodic and reactive intermittent execution,” in *PLDI ’20*, Jun. 2020, pp. 1005–1021.
- [7] B. Islam and S. Nirjon, “Scheduling Computational and Energy Harvesting Tasks in Deadline-Aware Intermittent Systems,” in *RTAS ’20*, Sep. 2020, pp. 95–109.
- [8] K. S. Yildirim *et al.*, “InK: Reactive Kernel for Tiny Batteryless Sensors,” in *SenSys ’18*, Nov. 2018, pp. 41–53.
- [9] A. Montanari *et al.*, “ePerceptive: energy reactive embedded intelligence for batteryless sensors,” in *SenSys ’20*, Nov. 2020, pp. 382–394.
- [10] B. Islam and S. Nirjon, “Zygard: Time-Sensitive On-Device Deep Inference and Adaptation on Intermittently-Powered Systems,” *IMWUT ’20*, vol. 4, no. 3, pp. 82:1–82:29, Sep. 2020.
- [11] L. L. Custode *et al.*, “Fast-Inf: Ultra-Fast Embedded Intelligence on the Batteryless Edge,” in *SenSys ’24*, Nov. 2024, pp. 239–252.
- [12] A. Bakar, T. Rahman, R. Shafik, F. Kawsar, and A. Montanari, “Adaptive Intelligence for Batteryless Sensors Using Software-Accelerated Tsetlin Machines,” in *SenSys ’22*, Jan. 2023, pp. 236–249.

- [13] C.-H. Yen, H. R. Mendis, T.-W. Kuo, and P.-C. Hsiu, "Stateful Neural Networks for Intermittent Systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4229–4240, Nov. 2022.
- [14] M. Xiang, P. M. Gholami, and H. Hoffmann, "Lupe: Integrating the Top-down Approach with DNN Execution on Ultra-Low-Power Devices," in *SensSys '25*, May 2025, pp. 357–370.
- [15] G. Park, O. Khan, and E. Seo, "Energy-Harvesting-Aware Adaptive Inference of Deep Neural Networks in Embedded Systems," in *ISLPED '23*, Aug. 2023, pp. 1–6.
- [16] F. Bambusi, F. Cerizzi, Y. Lee, and L. Mottola, "The Case for Approximate Intermittent Computing," in *IPSN '22*, May 2022, pp. 463–476.
- [17] L. Caronti, K. Akhunov, M. Nardello, K. S. Yildirim, and D. Brunelli, "Fine-grained Hardware Acceleration for Efficient Batteryless Intermittent Inference on the Edge," *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 5, pp. 82:1–82:19, Sep. 2023.
- [18] J. Choi, H. Joe, and C. Jung, "CapOS: Capacitor Error Resilience for Energy Harvesting Systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4539–4550, Nov. 2022.
- [19] Eaton, "Pm supercapacitors." [Online]. Available: <https://www.eaton.com/content/dam/eaton/products/electronic-components/resources/data-sheet/eaton-pm-supercapacitors-cylindrical-pack-data-sheet.pdf>
- [20] Yageo, "Supercapacitors ft series." [Online]. Available: https://www.yageogroup.com/content/datasheet/asset/file/KEM_S6014_FT
- [21] Kemet, "Supercapacitors fs series." [Online]. Available: https://content.kemet.com/datasheets/KEM_S6017_FS.pdf
- [22] H. Hu, D. Dey, M. Hebert, and J. A. Bagnell, "Learning anytime predictions in neural networks via adaptive loss balancing," in *AAAI'19/IAAI'19/EAAI'19*, Jan. 2019, pp. 3812–3821.
- [23] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," Sep. 2017, arXiv:1709.01686 [cs].
- [24] C. Wan, H. Hoffmann, S. Lu, and M. Maire, "Orthogonalized SGD and nested architectures for anytime neural networks," in *ICML'20*, vol. 119, Jul. 2020, pp. 9807–9817.
- [25] T. Kannan and H. Hoffmann, "Budget RNNs: Multi-Capacity Neural Networks to Improve In-Sensor Inference Under Energy Budgets," in *RTAS '21*, May 2021, pp. 143–156.
- [26] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, "Slimmable Neural Networks," Dec. 2018, arXiv:1812.08928 [cs].
- [27] J. Yu and T. Huang, "Universally Slimmable Networks and Improved Training Techniques," in *ICCV '19*, Oct. 2019, pp. 1803–1811.
- [28] B. Kutukcu, S. Baidya, and S. Dey, "SLEXNet: Adaptive Inference Using Slimmable Early Exit Neural Networks," *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 6, pp. 104:1–104:29, Sep. 2024.
- [29] C. Banbury *et al.*, "MLPerf Tiny Benchmark," Aug. 2021, arXiv:2106.07597 [cs].
- [30] E. Ruppel, M. Surbatovich, H. Desai, and B. Lucia, "An Architectural Charge Management Interface for Energy-Harvesting Systems," in *MI-CRO '22*, Oct. 2022, pp. 318–335.
- [31] A. Bakar, A. G. Ross, K. S. Yildirim, and J. Hester, "REHASH: A Flexible, Developer Focused, Heuristic Adaptation Platform for Intermittently Powered Computing," *IMWUT '21*, vol. 5, no. 3, pp. 87:1–87:42, Sep. 2021.
- [32] D. Balsamo *et al.*, "Hibernus++: A Self-Calibrating and Adaptive System for Transiently-Powered Embedded Devices," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 12, pp. 1968–1980, 2016.
- [33] N. A. Bhatti and L. Mottola, "HarvOS: Efficient Code Instrumentation for Transiently-Powered Embedded Sensing," in *IPSN '17*, Apr. 2017, pp. 209–220.
- [34] A. Colin and B. Lucia, "Chain: tasks and channels for reliable intermittent programs," in *OOPSLA '16*, Oct. 2016, pp. 514–530.
- [35] H. Jayakumar, A. Raha, and V. Raghunathan, "QUICKRECALL: A Low Overhead HW/SW Approach for Enabling Computations across Power Cycles in Transiently Powered Computers," in *2014 27th International Conference on VLSI Design and 2014 13th International Conference on Embedded Systems*, Jan. 2014, pp. 330–335.
- [36] K. Maeng, A. Colin, and B. Lucia, "Alpaca: intermittent execution without checkpoints," *OOPSLA '17*, vol. 1, pp. 96:1–96:30, Oct. 2017.
- [37] B. Ransford, J. Sorber, and K. Fu, "Mementos: system support for long-running computation on RFID-scale devices," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 1, pp. 159–170, Mar. 2011.
- [38] M. Surbatovich, L. Jia, and B. Lucia, "Automatically enforcing fresh and consistent inputs in intermittent systems," in *PLDI 2021*, Jun. 2021, pp. 851–866.
- [39] J. de Winkel, C. Delle Donne, K. S. Yildirim, P. Pawelczak, and J. Hester, "Reliable Timekeeping for Intermittent Computing," in *ASPLOS '20*, Mar. 2020, pp. 53–67.
- [40] J. Hester, K. Storer, and J. Sorber, "Timely Execution on Intermittently Powered Batteryless Sensors," in *SensSys '17*, Nov. 2017, pp. 1–13.
- [41] A. Colin, E. Ruppel, and B. Lucia, "A Reconfigurable Energy Storage Architecture for Energy-harvesting Devices," in *ASPLOS '18*, Mar. 2018, pp. 767–781.
- [42] K. Maeng and B. Lucia, "Adaptive dynamic checkpointing for safe efficient intermittent computing," in *OSDI'18*, Oct. 2018, pp. 129–144.
- [43] J. Van Der Woude and M. Hicks, "Intermittent computation without hardware support or programmer intervention," in *OSDI'16*, Nov. 2016, pp. 17–32.
- [44] C.-K. Kang, H. R. Mendis, C.-H. Lin, M.-S. Chen, and P.-C. Hsiu, "Everything Leaves Footprints: Hardware Accelerated Intermittent Deep Inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3479–3491, Nov. 2020.
- [45] —, "More Is Less: Model Augmentation for Intermittent Deep Inference," *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 5, pp. 49:1–49:26, Oct. 2022.
- [46] C. Wan, M. Santriai, E. Rogers, H. Hoffmann, M. Maire, and S. Lu, "ALERT: Accurate learning for energy and timeliness," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 353–369. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/wan>
- [47] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *ICML'15 - Volume 37*, Jul. 2015, pp. 448–456.
- [48] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.
- [49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 2015, arXiv:1512.03385.
- [50] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," Apr. 2018, arXiv:1804.03209.
- [51] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," Feb. 2018, arXiv:1711.07128 [cs, eess].
- [52] A. Chowdhery, P. Warden, J. Shlens, A. Howard, and R. Rhodes, "Visual Wake Words Dataset," Jun. 2019, arXiv:1906.05721 [cs, eess].
- [53] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks," Apr. 2017, arXiv:1704.04861 [cs].
- [54] Texas Instruments, "MSP430FR5994 Datasheet." [Online]. Available: <https://www.ti.com/lit/ds/symlink/msp430fr5994.pdf>
- [55] —, "Benchmarking the Signal Processing Capabilities of the Low-Energy Accelerator on MSP430 MCUs." [Online]. Available: <https://www.ti.com/lit/an/slaa698b/slaa698b.pdf>
- [56] PowerCast, "PowerCast-P2110B." [Online]. Available: <https://www.powercastco.com/products/rf-power/p2110b>
- [57] Digilent, "Analog Discovery 3 - USB Oscilloscope and Logic Analyzer." [Online]. Available: <https://digilent.com/shop/analog-discovery-3/>